Studio Nx

JS 288 — 트랙 믹서

2026-10-19

JavaScript 288 — 트랙 믹서 (`audio/mixer.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/mixer.js`
작업	S1-11a 재생 오디오 그래프: 트랙 볼륨, 팬, 뮤트, 솔로, 마스터, 이펙트 체인 자리 (FR-251, FR-252)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 287, 다음 289 클립 재생

1. 하는 일

트랙마다 "소스, 이펙트 체인, 볼륨, 팬" 순서로 노드를 이어 두고, 모든 트랙을 마스터로 모아 최종 출력으로 보냅니다. 뮤트와 솔로 규칙, dB 변환은 소리를 쓰지 않는 순수 함수로 따로 두었습니다. 컨텍스트를 인자로 받기 때문에 실시간 재생과 WAV 출력(S1-11b)이 같은 코드를 씁니다.

```
악기, 클립 ─▶ [source] ─▶ 이펙트 체인 ─▶ [input: 볼륨, 뮤트, 솔로] ─▶ [pan] ┐
                                                                              ├─▶ [master] ─▶ 마스터 이펙
트 ─▶ [out] ─▶ 스피커
(다른 트랙도 같은 모양) ──────────────────────────────────────────────────────┘
```

2. 언제 쓰나

쓰는 곳	하는 일
`PlaybackSession` (290)	재생을 시작할 때 믹서를 만들고, 재생 중에는 150ms마다 `applyTracks`를 불러 볼륨과 뮤트를 다시 반영합니다.
`ClipPlayer`, `InstrumentRack`	`trackInput(trackId)`으로 트랙의 입구 노드를 받아 소리를 보냅니다.
`EffectRack` (294)	`setTrackEffects`, `setMasterEffects`로 이펙트 체인을 바꿔 끼웁니다.
레벨 미터 (295)	`meterFor(trackId)`로 볼륨과 팬을 거친 소리의 크기를 읽습니다.

3. 코드

3.1 dbToGain

```
export function dbToGain(db) {
  const v = Number(db);
  if (!isFinite(v) || v <= -60) return 0;
  return Math.pow(10, v / 20);
}
```

3.2 audibleTrackIds

```
export function audibleTrackIds(tracks) {
  const list = (tracks || []).filter(function (t) { return t.track_type !== 'MASTER'; });
```

```
  const anySolo = list.some(function (t) { return !!t.is_solo; });
  return list.filter(function (t) {
    if (t.is_muted) return false;
    return anySolo ? !!t.is_solo : true;
  }).map(function (t) { return t.track_id; });
}
```

3.3 createStereoGain

```
export function createStereoGain(context) {
  const g = context.createGain();
  g.channelCount = 2;
  g.channelCountMode = 'explicit';
  g.channelInterpretation = 'speakers';
  return g;
}
```

3.4 MixerGraph 의 노드 만들기

```
constructor(context, destination) {
  this.context = context;
  this.master = context.createGain();          // 마스터 볼륨
  this.out = context.createGain();             // 마지막 출력 (마스터 이펙트 뒤) — 소리를 재거나 녹음할 때 여기에 붙임
  this.out.connect(destination != null ? destination : context.destination);
  this.master.connect(this.out);
  this.masterChain = [];                       // 마스터 이펙트 (master → 체인 → out)
  this.strips = {};   // trackId -> { source, input(볼륨), pan, chain }
  this.taps = {};     // 트랙 키 → LevelTap (미터)
}
```

```
strip(trackId) {
  const key = String(trackId);
  if (this.strips[key]) return this.strips[key];
  const ctx = this.context;
  const source = createStereoGain(ctx);   // 악기 · 클립이 모이는 곳 (이펙트 체인의 입구, 모노도 스테레오로 늘림)
  const input = ctx.createGain();      // 트랙 볼륨 · 뮤트 · 솔로 (이펙트 체인 뒤)
  const pan = typeof ctx.createStereoPanner === 'function' ? ctx.createStereoPanner() : null;
  source.connect(input);
  if (pan) { input.connect(pan); pan.connect(this.master); }
  else input.connect(this.master);
  this.strips[key] = { source: source, input: input, pan: pan, chain: [] };
  return this.strips[key];
}
```

```
trackInput(trackId) {
  return this.strip(trackId).source;
}
```

3.5 이펙트 체인 바꿔 끼우기

```
_rewire(head, tail, oldList, newList) {
  try { head.disconnect(); } catch (e) { /* 연결이 없었음 */ }
  oldList.forEach(function (fx) { try { fx.output.disconnect(); } catch (e) { /* 무시 */ } });
  let prev = head;
  newList.forEach(function (fx) { prev.connect(fx.input); prev = fx.output; });
  prev.connect(tail);
}
```

```
setTrackEffects(trackId, list) {
  const s = this.strip(trackId);
  this._rewire(s.source, s.input, s.chain, list);
  s.chain = list.slice();
}
```

```
setMasterEffects(list) {
  this._rewire(this.master, this.out, this.masterChain, list);
  this.masterChain = list.slice();
}
```

3.6 트랙 값 반영과 미터

```
applyTracks(tracks) {
  const audible = {};
  audibleTrackIds(tracks).forEach(function (id) { audible[String(id)] = true; });
  const self = this;
  const now = this.context.currentTime;
  (tracks || []).forEach(function (t) {
    if (t.track_type === 'MASTER') {
      self.master.gain.setTargetAtTime(dbToGain(t.volume_db), now, 0.01);
      return;
    }
    const s = self.strip(t.track_id);
    const gain = audible[String(t.track_id)] ? dbToGain(t.volume_db) : 0;
    s.input.gain.setTargetAtTime(gain, now, 0.01);   // 뚝 끊기는 소리(클릭)를 막으려고 아주 짧게 이어서
바꿈
    if (s.pan) s.pan.pan.setTargetAtTime(Math.max(-1, Math.min(1, Number(t.pan) || 0)), now, 0.01);
  });
}
```

```
meterFor(trackId) {
  const key = String(trackId);
  if (this.taps[key]) return this.taps[key];
  let node;
  if (key === 'master') node = this.out;
  else { const s = this.strip(key); node = s.pan || s.input; }
  this.taps[key] = new LevelTap(this.context, node);
  return this.taps[key];
}
```

4. 최신 문법으로 쓰면

실제 코드는 전통 문법으로 썼고, 최신 문법으로 쓰면 아래와 같습니다 (참고용입니다).

```
// audibleTrackIds 를 화살표 함수와 구조 분해로 쓴 모양
export const audibleTrackIds = (tracks = []) => {
  const list = tracks.filter(({ track_type }) => track_type !== 'MASTER');
  const anySolo = list.some(({ is_solo }) => !!is_solo);
  return list
    .filter(({ is_muted, is_solo }) => !is_muted && (!anySolo || !!is_solo))
    .map(({ track_id }) => track_id);
};
```

전통 문법 (실제 코드)	최신 문법
`function (t) { return ...; }`	`(t) => ...`
`tracks || []`	기본 매개변수 `tracks = []`
`t.track_type`	구조 분해 `{ track_type }`
`destination != null ? destination : context.destination`	`destination ?? context.destination`

5. 왜 이렇게

선택	이유
뮤트와 솔로 규칙을 순수 함수로 뺐습니다	소리 없이 Node에서 시험할 수 있고, 규칙이 바뀌어도 한곳만 고치면 됩니다.
트랙마다 `source`와 `input`을 따로 두었습니다	이펙트는 볼륨(페이더)보다 앞에 있어야 합니다. 그래서 `source`가 체인의 입구이고 `input`이 볼륨입니다.
마스터에 `master`와 `out`을 따로 두었습니다	마스터 이펙트(리미터 등)가 마스터 볼륨 뒤에 들어갑니다. 소리를 재거나 녹음할 때는 모든 처리가 끝난 `out`에 붙입니다.
입구를 스테레오로 고정했습니다 (`createStereoGain`)	모노 소리가 `StereoPannerNode`를 지나면 가운데에서 -3dB가 됩니다. 미리 스테레오로 늘려 두면 가운데에서는 크기가 그대로입니다. 자세한 내용은 문서 83을 봅니다.
값을 `setTargetAtTime`으로 바꿉니다	볼륨이 갑자기 바뀌면 "틱" 하는 잡음이 납니다. 10ms의 시간 상수로 부드럽게 잇습니다.
연결을 바꿀 때 `disconnect` 후 다시 잇습니다	노드 연결은 추가만 되고 덮어쓰기가 없어서, 옛 연결을 끊어야 소리가 두 번 나지 않습니다.
볼륨 -60dB 이하를 완전한 무음으로 둡니다	DB 범위의 아래 끝이 -60dB이고, 이 값을 "끔"으로 쓰기 때문입니다.

6. 시험

- `docs/handoff/s1-11a_tests/mixer_playback_unit.mjs`는 `dbToGain`과 `audibleTrackIds`를 확인합니다. 0dB는 1이고, -6dB는 약 0.5이며, -60dB는 0입니다. 솔로가 켜지면 솔로 트랙만 들리고, 뮤트가 솔로보다 우선하며, MASTER는 규칙에서 빠집니다.
- `docs/handoff/s1-11a_tests/ui.mjs`는 실제 화면에서 뮤트하면 무음이 되고, 해제하면 소리가 돌아오며, 다른 트랙이 솔로이면 이 트랙이 무음이 되는지 확인합니다.
- `docs/handoff/s1-11a_tests/ui_fx.mjs`는 마스터 트랙에 붙인 Limiter가 최종 출력의 피크를 줄이는지 확인합니다.